

Laboratorio 1 · Aprender a bisecar

Un agente que redescubre el método y demuestra que funciona

Métodos numéricos · Matemáticas · 90 minutos · en parejas, en los PCs del laboratorio

Antes de este laboratorio lee *La terminal desde cero*. Los primeros 30 minutos de la sesión son para hacer lo que dice esa guía, con ayuda. Todas las órdenes de abajo se escriben dentro de la carpeta `01_biseccion` y empiezan por `.venv\Scripts\python` (en Mac, `.venv/bin/python`).

Qué vas a hacer

Vas a mirar cómo un programa que **no conoce el método de bisección** lo redescubre solo, a base de premios y castigos. Después vas a romperlo a propósito para entender qué lo hace funcionar. Y al final vas a ver que «demostrar» el teorema de convergencia se puede plantear como un juego con reglas, y quién escribe las reglas.

Al terminar deberías poder explicar:

1. qué son *estado*, *acción*, *recompensa* y *política* en un problema de aprendizaje por refuerzo;
2. por qué la recompensa es la verdadera especificación del problema, y cómo se engaña a un agente;
3. qué demuestra de verdad el «agente que demuestra», y qué parte del trabajo es tuya.

1 Terminal e instalación

0-30 min

Sigue la guía de la terminal hasta el punto 6. Comprueba que estás listo con:

```
.venv\Scripts\python -m bisectrl --no-tui --seed 1
```

Debe terminar en un segundo e imprimir un informe. Si lo hace, levanta la mano para que el docente lo vea y sigue.

2 El juego

30-40 min, en papel

El método de bisección: f continua en $[a, b]$ con $f(a)f(b) < 0$. Se evalúa en el punto medio, se conserva la mitad donde f cambia de signo, se repite.

Ahora imagina que **no sabes** que hay que cortar en el medio ni cuál mitad conservar. En cada paso decides dos cosas:

- **dónde cortar:** una fracción λ del intervalo, entre 0,1 y 0,9;
- **qué lado conservar:** el izquierdo o el derecho.

Después de cada paso alguien te da puntos: -1 por cada evaluación de f (evaluar cuesta), más $\log_2(\text{ancho anterior}/\text{ancho nuevo})$ (premio por encoger el intervalo), y -10 con fin del juego si el intervalo que conservas ya no tiene cambio de signo.

Eso es un problema de **aprendizaje por refuerzo** (RL): un *agente* observa un *estado*, elige una *acción*, recibe una *recompensa*, y repite miles de veces hasta que su *política* (qué hace en cada estado) es buena. No le dicen qué es «bueno»; solo le dan puntos.

Responde antes de correr nada

- (a) Si cortas siempre en $\lambda = 0,5$, ¿cuántos puntos ganas por paso?
- (b) ¿Basta ver los **signos** de $f(a)$, $f(x)$, $f(b)$ para decidir qué lado conservar?
- (c) Predice: ¿qué λ va a preferir el agente? ¿Por qué?

3 Mira cómo aprende

40–55 min

```
.venv\Scripts\python -m bisectrl --seed 1
```

Observa los paneles (la ventana debe estar maximizada). En la bitácora aparecen los hitos. Anota la generación de cada uno:

hito	gen. (seed 1)	gen. (seed 2)
primera convergencia		
«aprendió el invariante de Bolzano»		
«descubrió $\lambda = 1/2$ »		
primera demostración completa		
primera demostración sin saltos lógicos		

Al terminar se guarda `runs\<fecha>\informe.md`. Ábrelo con el Bloc de notas y lee la sección «**Síntesis**».

- (d) En la tabla $Q(\lambda)$, ¿qué significa que $Q(0,5) \approx 0$ y $Q(0,1) \approx -0,6$? Relaciónalo con (a).
- (e) Corre otra vez con `--seed 2` y completa la segunda columna. ¿Cambian las generaciones de los hitos? ¿Cambia lo aprendido?

4 Rómpelo: la recompensa es la especificación

55–70 min

El agente no sabe qué es «bisecar bien». Solo sabe qué le da puntos. Vamos a cambiar los puntos.

```
.venv\Scripts\python ..\docs\curso\lab01\experimentos.py recompensa
```

Corre tres entrenamientos: el original, uno donde **perder la raíz cuesta 0** en vez de -10 , y uno donde **cada paso es gratis**.

- (f) En el caso «perder la raíz cuesta 0», ¿cuántas veces converge el agente? ¿Qué aprendió a hacer? Explica **por qué es racional** desde el punto de vista de los puntos. (Pista: compara lo que gana un episodio que pierde la raíz en el primer paso con uno que biseca 12 veces.)
- (g) En el caso «cada paso es gratis», el agente sigue eligiendo $\lambda = 1/2$. ¿Por qué?
- (h) Propón otra recompensa que **parezca** razonable y que el agente pueda explotar sin resolver el problema. Descríbela y di qué haría el agente.

Si te sobra tiempo:

```
.venv\Scripts\python ..\docs\curso\lab01\experimentos.py sin_medio
```

- (i) Sin la opción 0,5, ¿qué λ elige? Dibuja la curva $Q(\lambda)$.

5 La demostración como grafo

70-85 min

Abre `bisectrl\proof_kb.py` con el Bloc de notas. El teorema de convergencia de la bisección está escrito como **14 pasos con dependencias** (deps) más **7 distractores**.

- (j) Dibuja en papel el grafo: un nodo por paso, una flecha desde cada dependencia hacia el paso que la usa. Marca las hipótesis (sin flechas de entrada) y ■.
- (k) El distractor `D_B0LZ` dice: «por Bolzano existe una raíz, luego el método converge a ella». ¿Qué le falta?
- (l) Compara tu grafo con la demostración del agente en `runs\<fecha>\demostracion.md`. ¿El agente «entendió» la demostración? ¿Qué es lo que realmente aprendió?

Ahora el experimento central del laboratorio:

```
.venv\Scripts\python ..\docs\curso\lab01\experimentos.py grafo
```

Quitamos una sola dependencia: el paso `R00T` (« $f(c)^2 \leq 0$, luego $f(c) = 0$ ») deja de exigir el invariante `INV`. El verificador acepta una demostración de 13 pasos que **nunca prueba** que el cambio de signo se conserva.

- (m) ¿Dónde está el hueco? Escribe con tus palabras por qué $f(c)^2 \leq 0$ **no se sigue** sin `INV`.
- (n) Conclusión: el agente encuentra un orden válido *para el grafo que le dieron*. ¿Quién es responsable de que la demostración sea correcta?

6 Cierre

85-90 min

Entrega (una por pareja, en Word o en texto, antes de la próxima sesión):

1. La tabla de hitos con dos semillas (página 2).
2. Las respuestas a (a)-(n) en la hoja de entrega de las páginas siguientes. Una respuesta correcta suele caber en dos líneas.
3. El párrafo final de la hoja de entrega. Esto es la semilla del proyecto final.

Puedes entregar esta misma hoja escrita a mano, o el mismo contenido en Word.

Tarea opcional

Añade en `proof_kb.py` un distractor propio: un paso que **tú** habrías escrito en un examen y que está mal. Basta una línea como `Step("D_MIO", "Distractor", "texto", distractor=True)` dentro de la lista `STEPS`. Corre con `--no-tui` y busca en el informe cuántas veces lo intentó el agente.

Hacia dónde va esto. En las próximas sesiones el agente aprenderá punto fijo, Taylor, Newton, interpolación, integración, sistemas lineales y ecuaciones diferenciales con la misma receta. Luego **tú** escribirás el grafo de un teorema nuevo y las recompensas de un método. Y al final el verificador no será un grafo escrito a mano sino un programa que certifica cada propuesta: ahí el agente sí puede encontrar cosas que nadie le dijo.

Hoja de entrega · Laboratorio 1

Nombres: _____

Fecha: _____ Semillas usadas: _____

(a) puntos por paso con $\lambda = 0,5$

(b) ¿bastan los signos?

(c) predicción de λ y por qué

(d) qué significan $Q(0,5) \approx 0$ y $Q(0,1) \approx -0,6$

(e) con otra semilla, ¿qué cambia y qué no?

(f) perder la raíz cuesta 0: cuántas convergencias, qué aprendió, por qué es racional

(g) paso gratis: ¿por qué sigue eligiendo $\lambda = 1/2$?

(h) una recompensa que parece razonable y cómo la explotaría el agente

(i) sin la opción 0,5: qué elige, y dibujo de $Q(\lambda)$

(j) el grafo de la demostración (nodos, flechas, hipótesis marcadas, ■)

(k) qué le falta al distractor «por Bolzano existe raíz, luego el método converge»

(l) ¿el agente entendió la demostración? ¿qué aprendió realmente?

(m) dónde está el hueco sin INV

(n) ¿quién es responsable de que la demostración sea correcta?

Párrafo final: si escribieras el grafo de otro teorema del curso, ¿cuál, con qué hipótesis y qué ■?